Call Me Maybe

Cours 7 — Structure du projet et qualite du code

---

1. Structure des fichiers attendue

```
mon_projet/
■■■ src/
■   ■■■ __init__.py
■   ■■■ __main__.py      # point d'entree : uv run python -m src
■   ■■■ generator.py     # boucle de generation + constrained decoding
■   ■■■ models.py        # classes Pydantic
■   ■■■ loader.py        # chargement des fichiers JSON
■   ■■■ vocabulary.py    # chargement du vocabulaire
■■■ llm_sdk/
■   ■■■ __init__.py      # copie du package fourni
■■■ data/
■   ■■■ input/
■   ■   ■■■ functions_definition.json
■   ■   ■■■ function_calling_tests.json
■   ■■■ output/          # genere au runtime, pas dans git
■■■ pyproject.toml
■■■ uv.lock              # genere par uv sync
■■■ Makefile
■■■ README.md
■■■ .gitignore
```

2. Le pyproject.toml

torch et transformers ne sont PAS dans tes dependances — ils viennent du llm_sdk que tu copies directement. Le reviewer fait uv sync puis copie le llm_sdk.

```
[project]
name = "call-me-maybe"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "numpy",
    "pydantic",
]
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
[tool.hatch.build.targets.wheel]
packages = ["src"]
[tool.mypy]
python_version = "3.10"
warn_return_any = true
warn_unused_ignores = true
disallow_untyped_defs = true
check_untyped_defs = true
ignore_missing_imports = true
```

3. Le Makefile — regles imposees par le sujet

```
.PHONY: install run debug clean lint lint-strict
install:
uv sync
run:
uv run python -m src
debug:
uv run python -m pdb -m src
clean:
find . -type d -name __pycache__ -exec rm -rf {} +
find . -type d -name .mypy_cache -exec rm -rf {} +
find . -type f -name '*.pyc' -delete
lint:
uv run flake8 .
uv run mypy . \
--warn-return-any \
--warn-unused-ignores \
--ignore-missing-imports \
--disallow-untyped-defs \
--check-untyped-defs
lint-strict:
uv run flake8 .
uv run mypy . --strict
```

Attention : le Makefile utilise des tabulations, pas des espaces. Un espace a la place d'une tabulation = erreur de syntaxe make.

4. Les type hints — ce que mypy verifie

```
# ■■ Fonctions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def load_function_definitions(path: str) -> list[FunctionDef]:
    ...
def get_valid_tokens(
    ctx: GenContext,
    vocab: dict[str, int],
) -> list[int]:
    ...
def apply_mask(
    logits: list[float],
    valid_ids: list[int],
) -> np.ndarray:
    ...
# ■■ Variables locales ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
vocab: dict[str, int] = {}
func_tokens: dict[str, list[int]] = {}
results: list[FunctionCall] = []
# ■■ Attributs de dataclass ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@dataclass
class GenContext:
    state        : JsonState
    chosen_func  : FunctionDef | None = None   # Python 3.10+
    params_done  : list[str] = field(default_factory=list)
    current_param: str | None = None
    current_value: str = ''
```

5. Les docstrings — style Google (PEP 257)

```
def get_valid_tokens(
    ctx: GenContext,
    vocab: dict[str, int],
) -> list[int]:
    """Retourne les IDs des tokens valides pour l'etat courant.
    Args:
        ctx: Le contexte de generation contenant l'etat courant.
        vocab: Le vocabulaire du tokenizer (token -> ID).
    Returns:
        Liste des IDs de tokens autorises a cette position.
    Raises:
        ValueError: Si aucun token valide n'existe pour l'etat.
    """
    ...
def load_function_definitions(path: str) -> list[FunctionDef]:
    """Charge et valide les definitions de fonctions depuis un JSON.
    Args:
        path: Chemin vers le fichier functions_definition.json.
    Returns:
        Liste de FunctionDef validees par Pydantic.
    Raises:
        FileNotFoundError: Si le fichier est introuvable.
        ValueError: Si le JSON est invalide ou le schema incorrect.
    """
    ...
```

6. Le __main__.py — point d'entree

Tout le code est dans main() — pas directement sous if __name__. Raison : ca permet d'importer main() dans un test sans executer le programme.

```
"""Point d'entree du programme call me maybe."""
import argparse
import sys
from src.loader import load_function_definitions, load_test_prompts
from src.generator import generate_all
def parse_args() -> argparse.Namespace:
    """Parse les arguments de la ligne de commande.
    Returns:
        Namespace contenant les chemins des fichiers.
    """
    parser = argparse.ArgumentParser(description="Function calling")
    parser.add_argument(
        "--functions_definition",
        default="data/input/functions_definition.json",
    )
    parser.add_argument(
        "--input",
        default="data/input/function_calling_tests.json",
    )
    parser.add_argument(
        "--output",
        default="data/output/function_calling_results.json",
    )
    return parser.parse_args()
def main() -> None:
    """Fonction principale du programme."""
    args = parse_args()
    try:
        generate_all(
            functions_path=args.functions_definition,
            input_path=args.input,
            output_path=args.output,
        )
    except Exception as e:
        print(f"Erreur fatale : {e}", file=sys.stderr)
        sys.exit(1)
if __name__ == "__main__":
    main()
```

7. Le .gitignore

```
# Python
__pycache__/
*.pyc
*.pyo
.mypy_cache/
.venv/
*.egg-info/
# Projet — ne pas versionner les sorties
data/output/
# IDE
.vscode/
.idea/
```

8. flake8 — erreurs courantes a eviter

Code	Erreur	Mauvais	Correct
E501	Ligne > 79 caracteres	`def f(a,b,c,d,e,f): return a+b`	`def f( a: int, b: int, ) -> int:`
E302	2 lignes vides entre fonctions	`def foo(): pass def bar(): pass`	`def foo(): pass def bar(): pass`
E711	Comparaison None avec ==	`if x == None:`	`if x is None:`
F401	Import inutilise	`import os # jamais utilise`	`# supprimer la ligne`
W293	Espaces en fin de ligne	`x = 1 <- espaces cachés`	`x = 1`

9. uv sync — comment ca fonctionne

uv sync lit uv.lock (versions exactement figees), cree le venv si absent, et installe tout en une commande. C'est reproductible : tout le monde obtient exactement les memes versions. pip install ne garantit pas ca.

```
# Workflow complet pour demarrer le projet
git clone <ton-repo>
cd mon_projet
cp -r /chemin/llm_sdk ./llm_sdk   # copier le SDK fourni
uv sync                            # installe numpy + pydantic
make run                           # lance le programme
# Pour verifier la qualite du code
make lint                          # flake8 + mypy
make lint-strict                   # mypy --strict (optionnel)
# Pour nettoyer
make clean                         # supprime __pycache__ et .mypy_cache
```

Exercice

`Q1. Reecris cette fonction pour qu'elle passe mypy et flake8 : def process(data, functions, vocab): results = [] for item in data: r = generate(item, functions, vocab) results.append(r) return results`

Q2. Que fait exactement uv sync ? Quelle est la difference avec pip install ?

Q3. Dans __main__.py, pourquoi met-on tout dans main() plutot que directement sous if __name__ == '__main__' ?

Reponses

Q 1	Ajouter les type hints sur les parametres et le retour, une docstring Google style, et typer la variable results : def process(data: list[TestPrompt], functions: list[FunctionDef], vocab: dict[str, int],) -> list[FunctionCall]: """Traite tous les prompts...""" results: list[FunctionCall] = [] for item in data: results.append(generate(item, functions, vocab)) return results
Q 2	uv sync lit uv.lock (versions exactement figees), cree le venv si absent, et installe tout automatiquement. pip install ne garantit pas la reproductibilite — deux personnes peuvent obtenir des versions differentes selon leur environnement.
Q 3	Mettre le code dans main() permet de l'importer depuis un test sans l'executer (from src.__main__ import main). Si le code est directement sous if __name__, il s'execute au moment de l'import et casse les tests unitaires. C'est aussi plus lisible et plus facile a tester.

A retenir

- src/__main__.py est le point d'entree : uv run python -m src.
- torch et transformers ne sont pas dans pyproject.toml — ils viennent du llm_sdk copie.
- Le Makefile utilise des tabulations (pas des espaces).
- Type hints sur tous les parametres et retours — mypy ne pardonne pas.
- Docstrings Google style sur toutes les fonctions et classes.
- data/output/ dans .gitignore — le dossier est genere au runtime.

Prochaine etape

Cours 8 — Assemblage final : tout mettre ensemble, creer les fichiers un par un, tester avec les 11 prompts reels, gerer les edge cases et rediger le README.